

Continual Reinforcement Learning for Evolving Games: Learning Across Game Updates in Competitive Pokémon

Thomas Vroom



Maastricht University



Content

- Introduction
- Competitive Pokémon
- Preliminary Experiments & Results
- Methods
- Experiments & Results
- Conclusion

Introduction

Introduction

- **Reinforcement Learning (RL)** can achieve superhuman performance in sequential decision-making tasks.
- Two phases: *Training* and *Deployment* → **Stationarity Assumption**
 - Is it realistic? *Autonomous driving, recommender systems, games...*
- Modern games are *live-service*: they receive frequent balance patches, content updates, seasonal events, etc.
 - *Keep using the same strategy?* → Unpredictable performance
 - *Retrain after every update?* → Expensive training costs
 - *Transfer knowledge across game versions?*

Introduction

- **Continual (Reinforcement) Learning:** adapt to a sequence of tasks over time without forgetting previously learned knowledge.
- *Stability:* preservation of previously acquired knowledge.
- *Plasticity:* ability to learn new information.
- **Catastrophic forgetting:** learning new information degrades performance on previously learned tasks.
- **Loss of plasticity:** networks gradually lose their ability to adapt to new tasks or changing data over time.

Introduction

- We investigate whether continual reinforcement learning can facilitate effective knowledge transfer in a non-stationary, competitive game.
- Our domain is *competitive Pokémon*, also known as **VGC**.
- Legal Pokémon change under seasonal rulesets.
 - Intuitive example of non-stationarity.
 - Clear opportunity for knowledge transfer.
 - Clear task boundaries.

<i>Regulation</i>	<i>#Legal Pokémon</i>
A	380
B	394
C	398
D	494
E	593
F	698
G	718
H	631
I	718
J	733

Introduction - Research Questions

1. What effect does non-stationarity have on the performance and generalization of reinforcement learning agents in VGC?
2. Can continual reinforcement learning methods facilitate effective knowledge transfer across game versions?
3. How do continual reinforcement learning-based agents compare to fully retrained agents in terms of performance, generalization, and training cost across game updates?

Competitive Pokémon

Competitive Pokémon

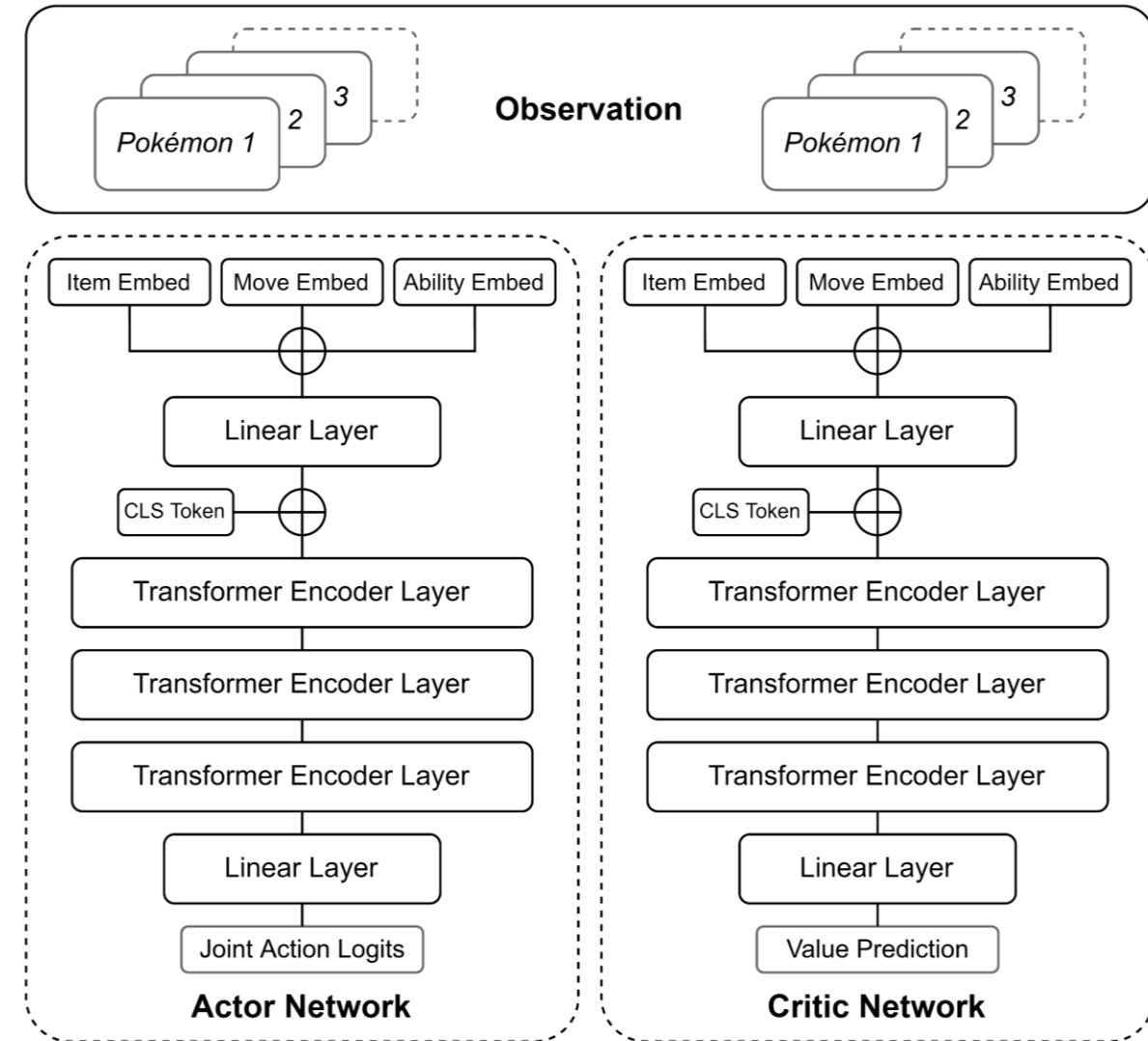
- VGC consists of two phases: team building and *battling*.
- Players assemble a team of six (distinct) Pokémon, customizing their:
 - *HP, Attack, Special Attack, Defense, Special Defense, Speed*
 - *Ability, nature, held item, and up to four (distinct) moves*
- Number of possible teams: 10^{139} (estimated¹)
- Most of this is not visible to the opponent → **Partial Observability**
- Size of the information set: 10^{58} (estimated lower-bound¹)

Competitive Pokémon

- VGC consists of two phases: *team building* and *battling*.
- A player has two Pokémon active at once (two more to switch in later).
- Battles play out in turns, but players make moves simultaneously:
 - Attack one or both opponent's Pokémon.
 - Switch one or both Pokémon out.
 - Select a utility move.
- Move outcomes involve a large amount of random chance:
 - Miss target, secondary effects, status conditions, critical hits, etc.
- Objective: knock out all the opponent's Pokémon (*two-player zero-sum*)

Competitive Pokémon

- Training RL agents on VGC → **VGC-Bench**¹
 - Multi-agent RL environment for VGC.
 - Meta-relevant teams for each regulation.
 - Various baseline agents, incl. *fine-tuned behavior cloning* and *self-play RL*.
- All agents trained using *Proximal Policy Optimization (PPO)*.
 - *In-distribution set*: teams seen during training.
 - *Out-of-distribution set*: teams not seen during training.
- Fine-tuned behavior cloning + Less teams seen during training performs best.



Preliminary Experiments

Preliminary Experiments

- Problem formalization:
 - Let A_t be an agent trained on version t of game G_t for n steps.
 - Can A_t outperform A_{t+1} on game G_{t+1} in at most n additional steps?

Assumption 1: t is monotonically increasing.

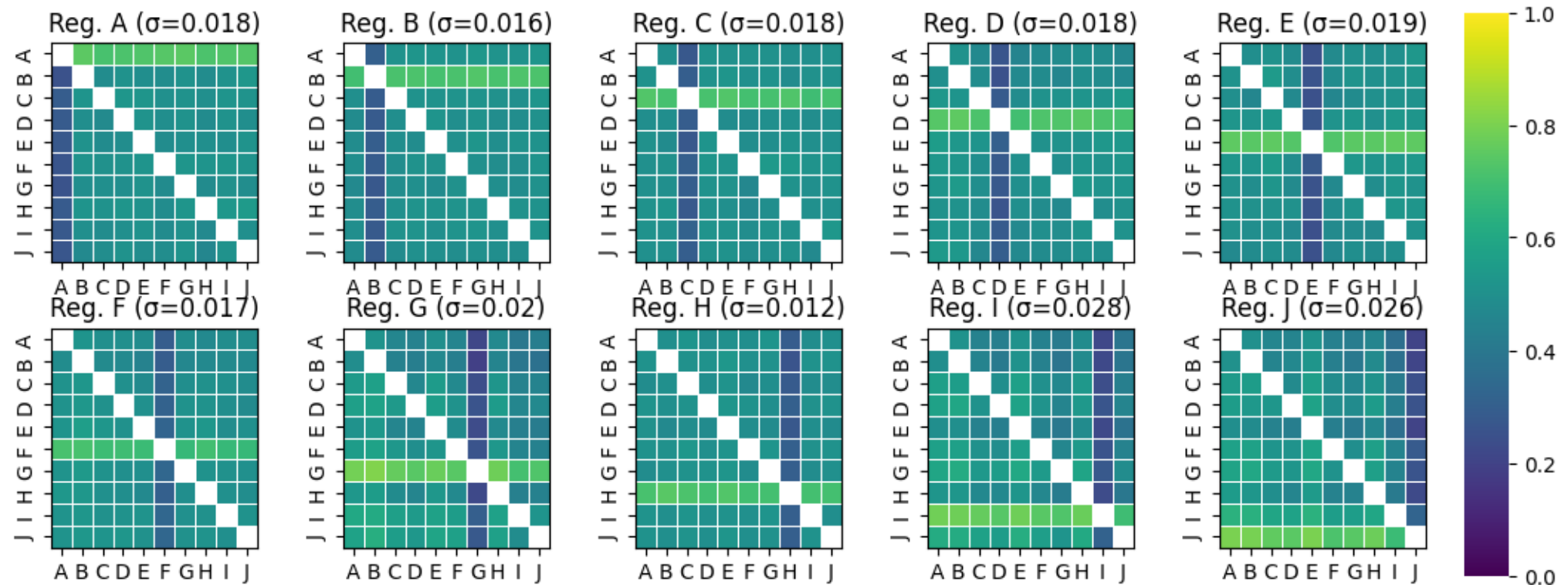
Assumption 2: G_{t+1} shares its action space and reward function with G_t .

Assumption 3: No information about G_{t+1} is available a priori.

- For VGC: Train on regulation $\mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{C} \rightarrow \dots$
 - A1: no regulation is revisited.
 - A2: domain-incremental learning.
 - A3: cannot use fine-tuned behavior cloning $\rightarrow$ self-play RL
 - 64 teams seen during training $\rightarrow$ Highest chance of successful transfer

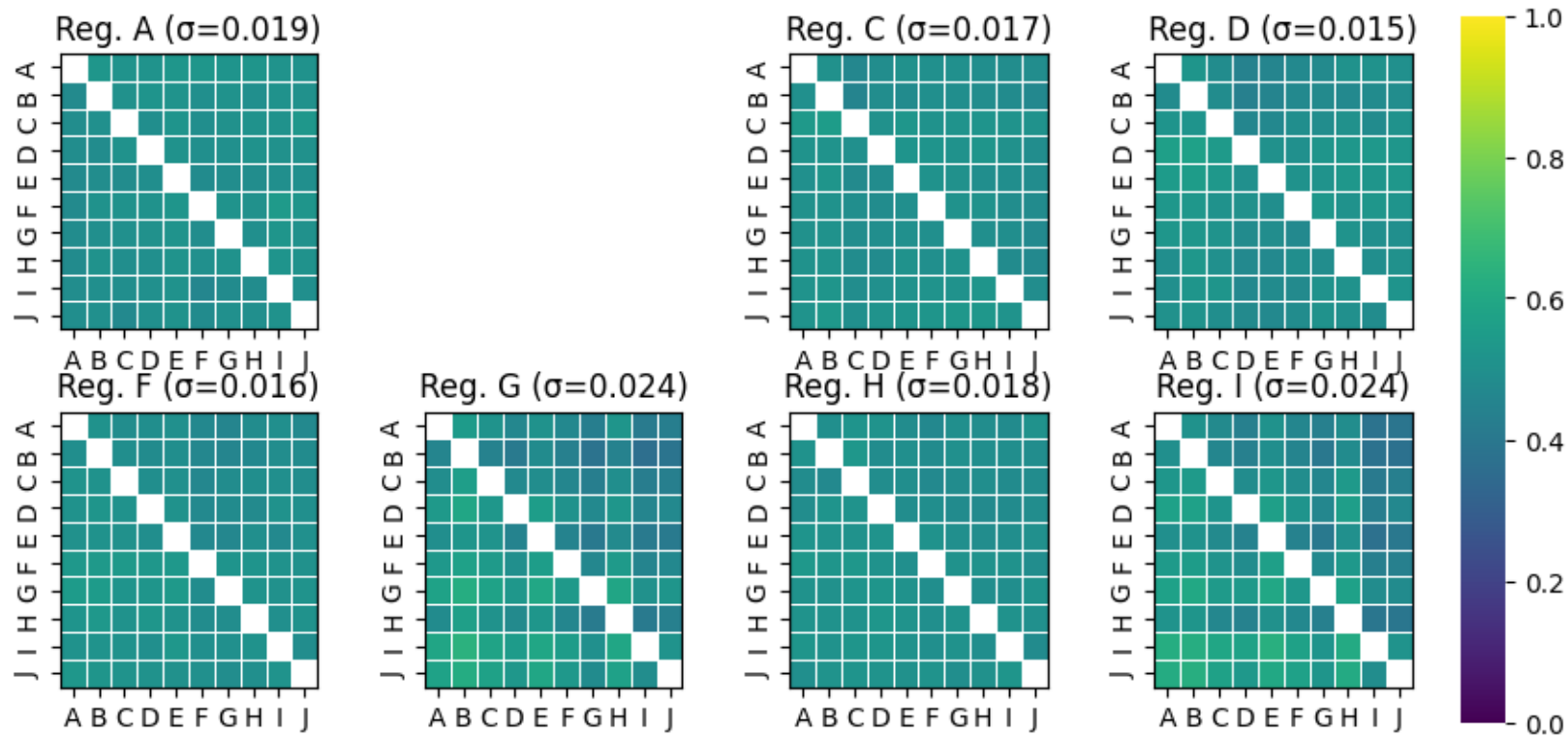
Preliminary Experiments - Zero-Shot Transfer

1. What effect does non-stationarity have on the performance and generalization of reinforcement learning agents in VGC?



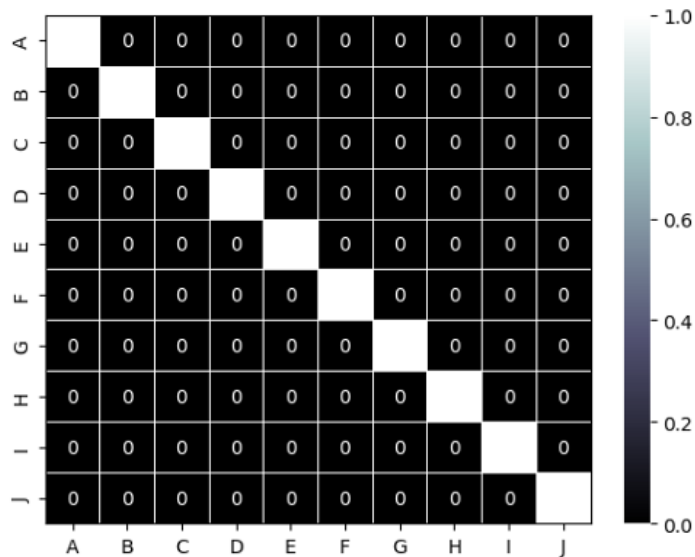
Preliminary Experiments - Zero-Shot Transfer

1. What effect does non-stationarity have on the performance and generalization of reinforcement learning agents in VGC?

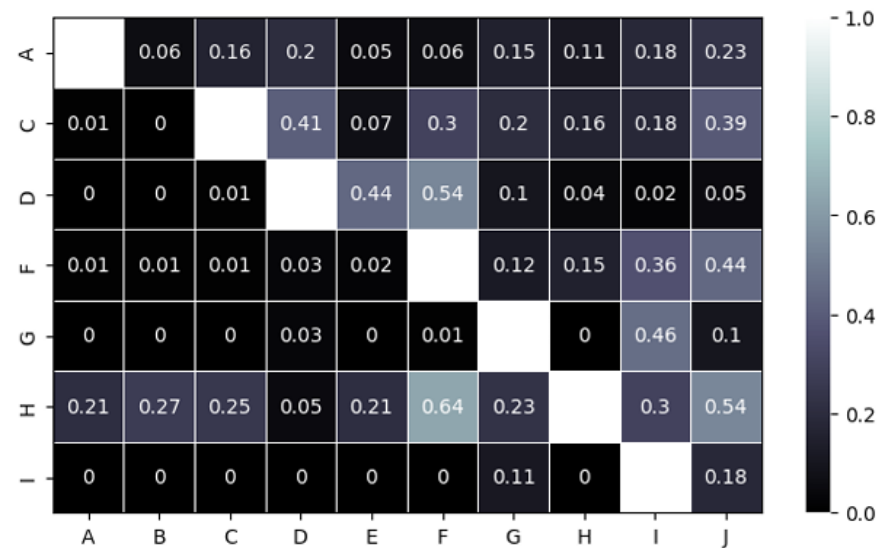


Preliminary Experiments - Zero-Shot Transfer

- H_0 = An agent trained on regulation $\tau \neq t$ performs *worse* in regulation t
- $\rightarrow$ **Binomial Test:** $n = 1000, p < 0.5, \alpha = 0.05$



(a) In-distribution.

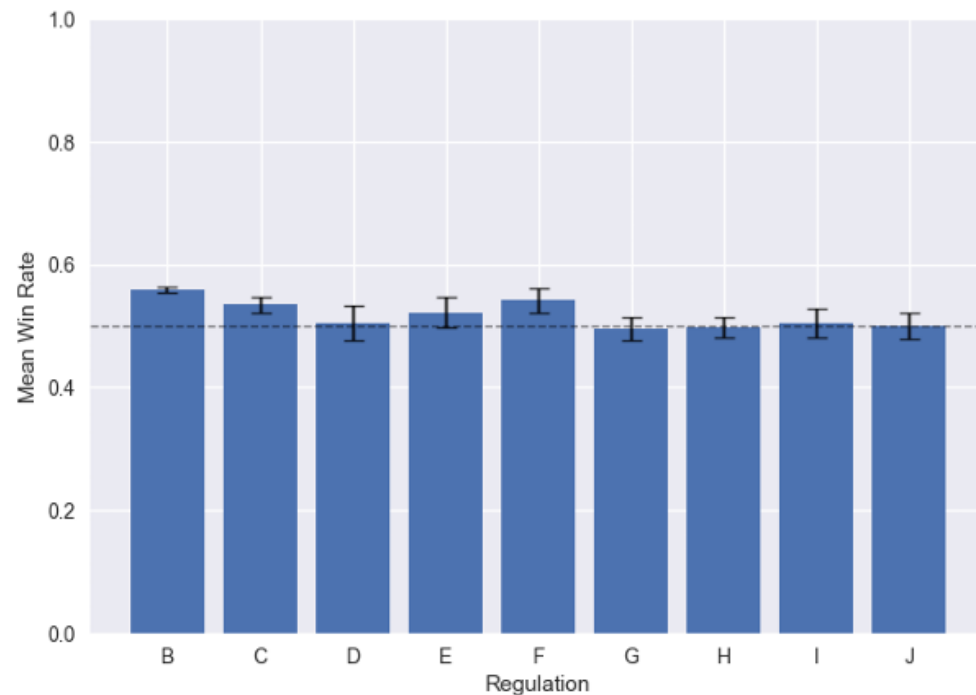


(b) Out-of-distribution.

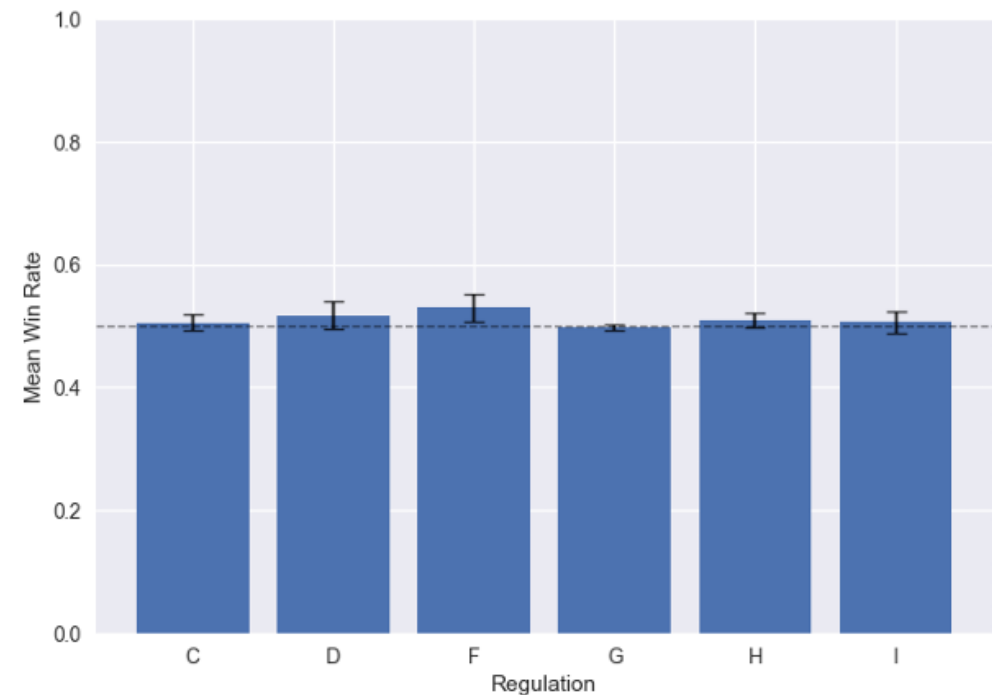
- Generally speaking, RL agents are capable of zero-shot transfer to future regulations when neither agent has seen the teams before, but not in any other scenario.

Preliminary Experiments - Full Fine-Tuning

2. To what extent are reinforcement learning agents able to maintain performance under non-stationarity without any continual learning methods.



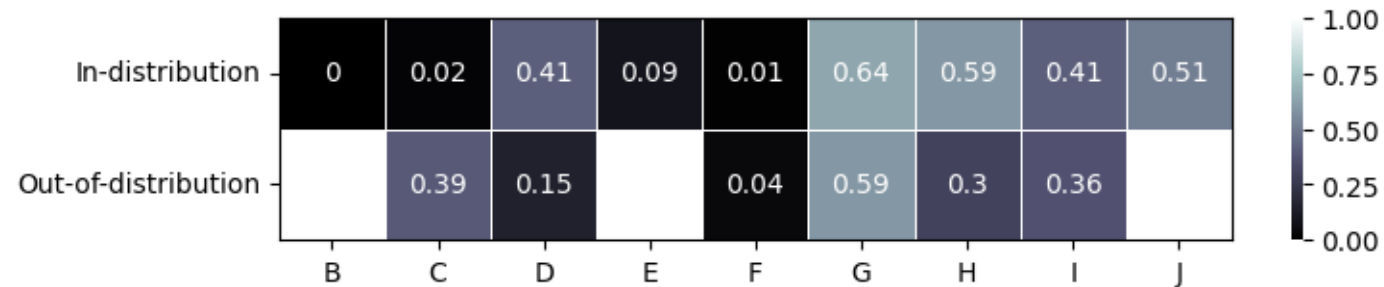
(a) In-distribution.



(b) Out-of-distribution.

Preliminary Experiments - Full Fine-Tuning

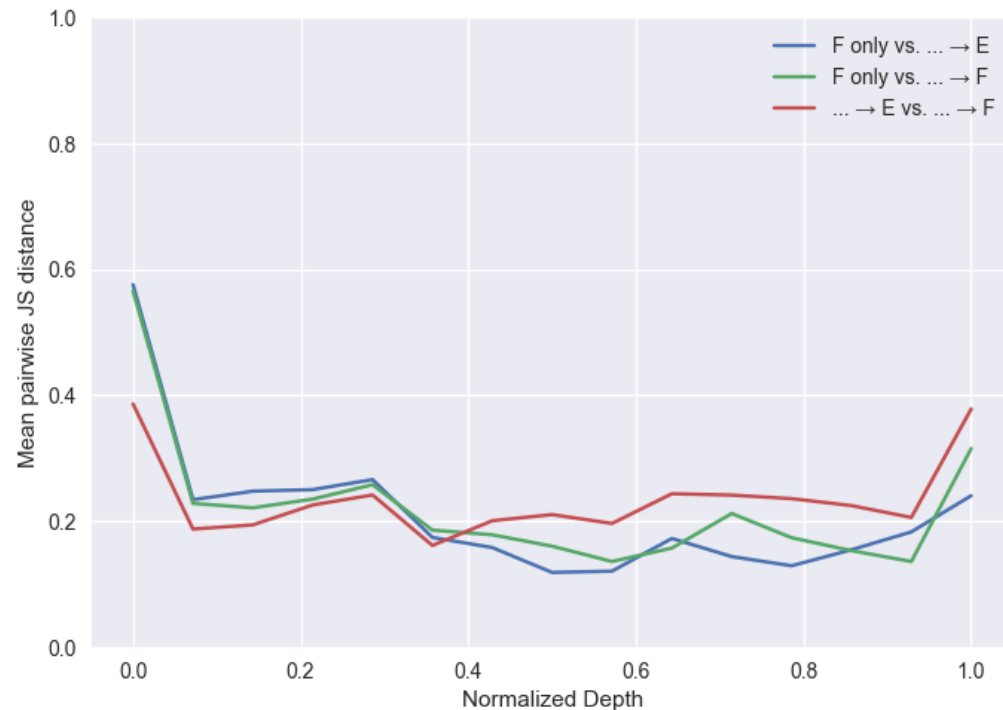
2. To what extent are reinforcement learning agents able to maintain performance under non-stationarity without any continual learning methods.
- H_0 = An agent ϕ performs *equal or worse* than agent ψ
 - → **Binomial Test**: $n = 1000, p \leq 0.5, \alpha = 0.05$



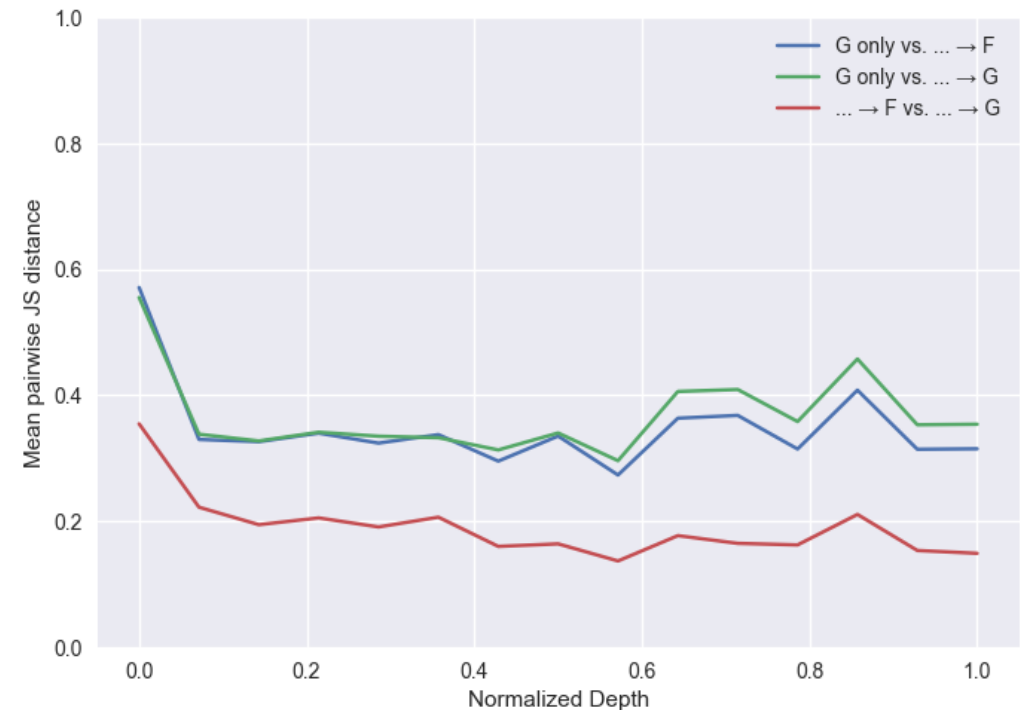
- In the best case, RL agents are capable of positive transfer short-term.

Preliminary Experiments - Analysis

- Mean pairwise Jensen-Shannon distance between action probabilities:



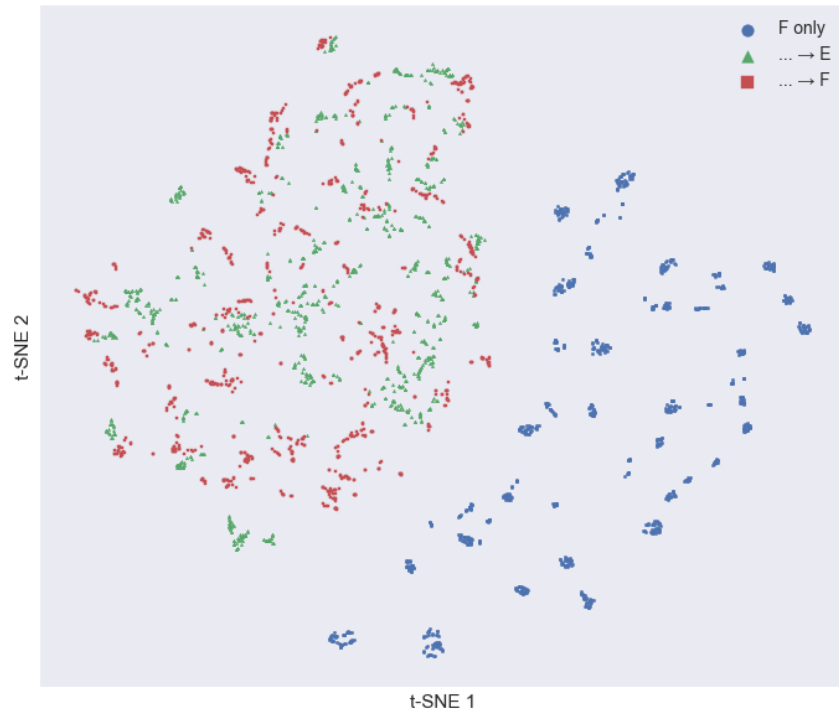
(Regulation **F**, in-distribution)



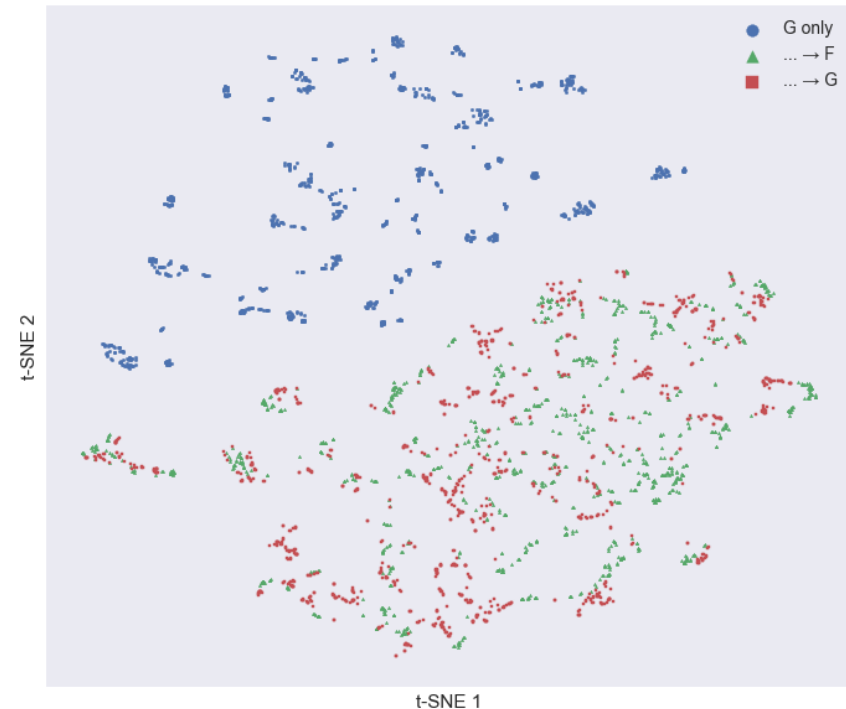
(Regulation **G**, in-distribution)

Preliminary Experiments - Analysis

- Distributions of action logits (t-SNE):



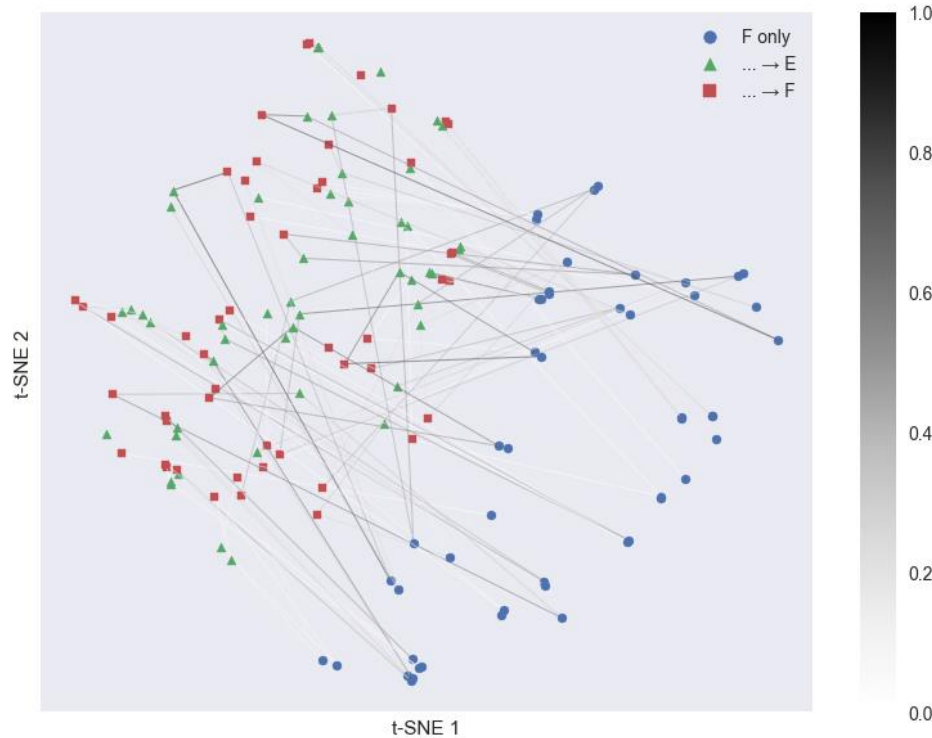
(Regulation **F**, in-distribution)



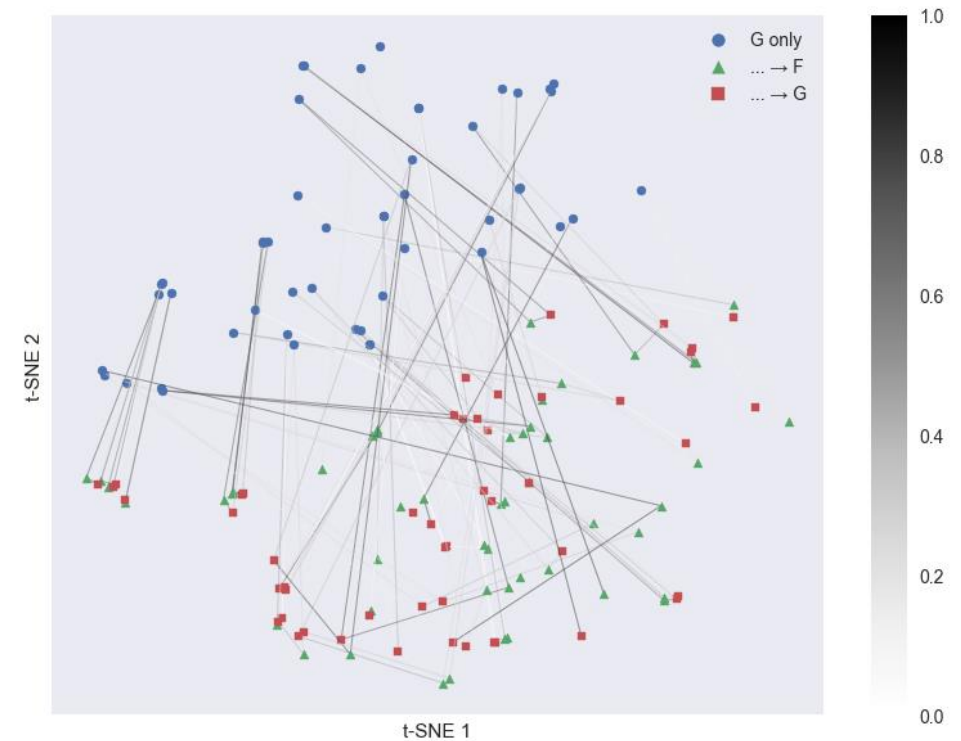
(Regulation **G**, in-distribution)

Preliminary Experiments - Analysis

- Distributions of action logits (t-SNE), Jensen-Shannon distance (lines):



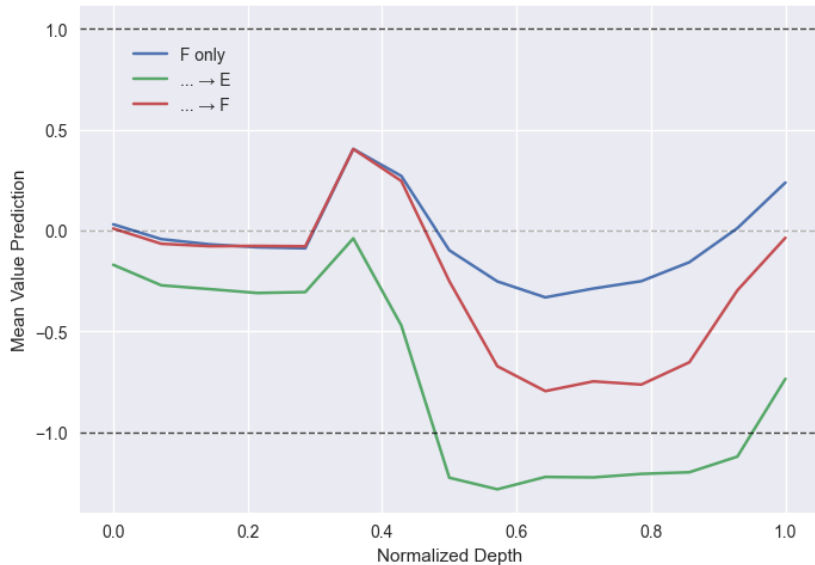
(Regulation **F**, in-distribution)



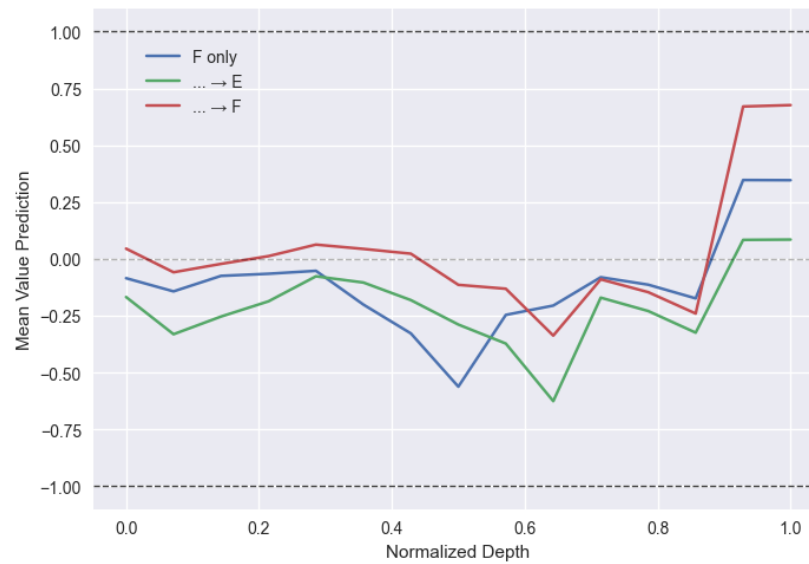
(Regulation **G**, in-distribution)

Preliminary Experiments - Analysis

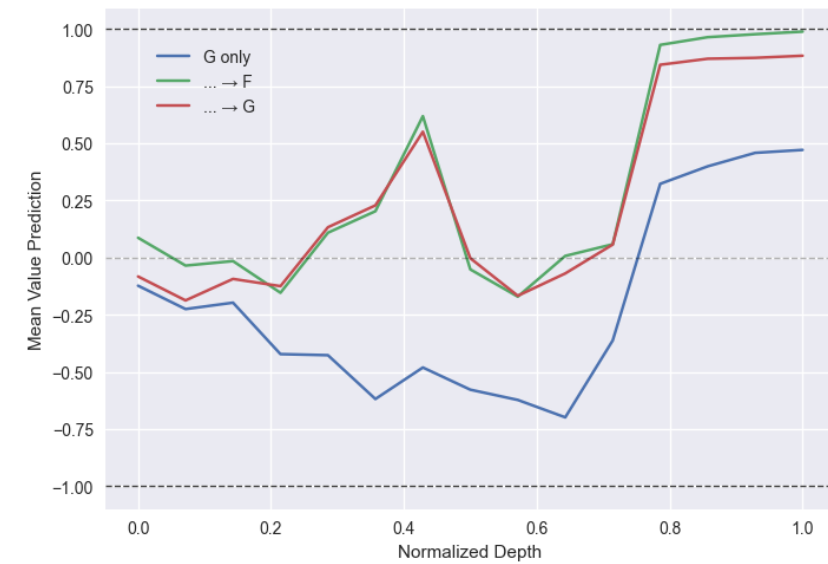
- Mean value prediction:



(Regulation **F**, in-distribution)



(Regulation **F**, out-of-distribution)

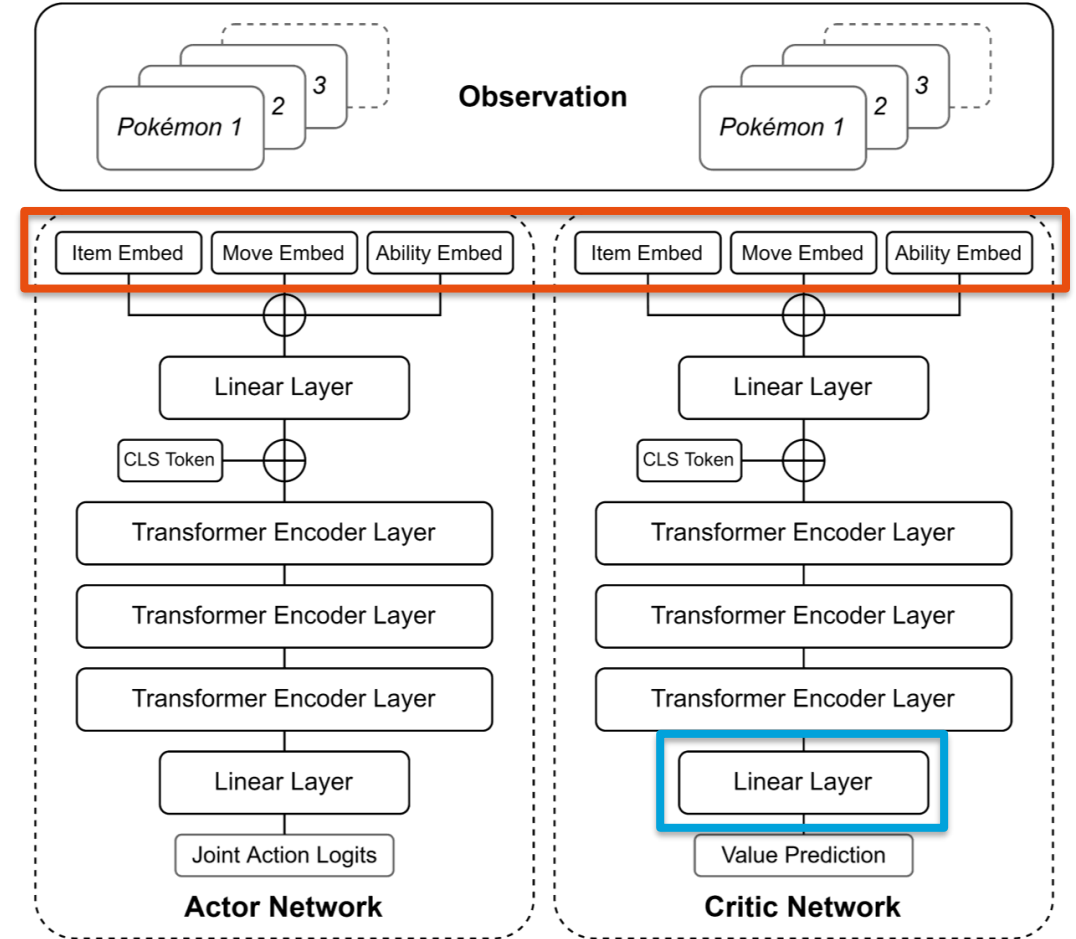


(Regulation **G**, out-of-distribution)

Methods

Methods - Value Stability

- We propose two adjustments to VGC-Bench's policy architecture:
- Initialization of **embeddings**:
 $\mathcal{N}(0,1) \rightarrow \mathcal{N}(0,0.02)$
 - Initialization of **value projection**:
 $\text{Gain} = 1 \rightarrow \text{Gain} = 0.01$



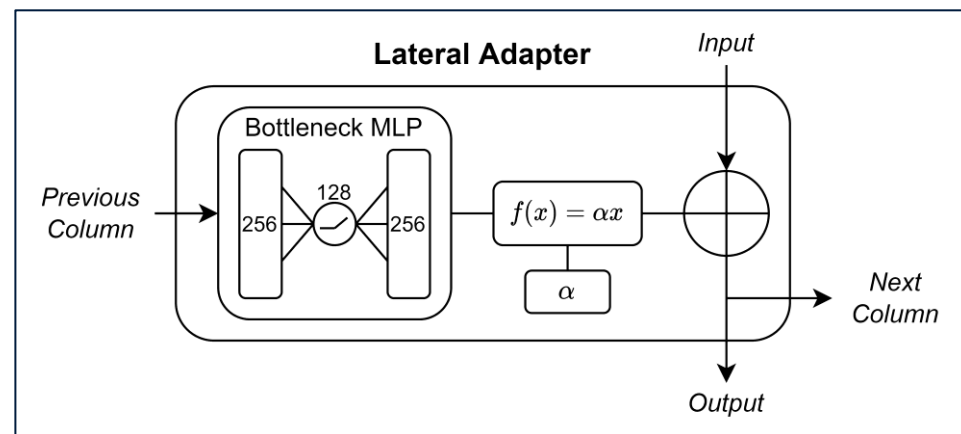
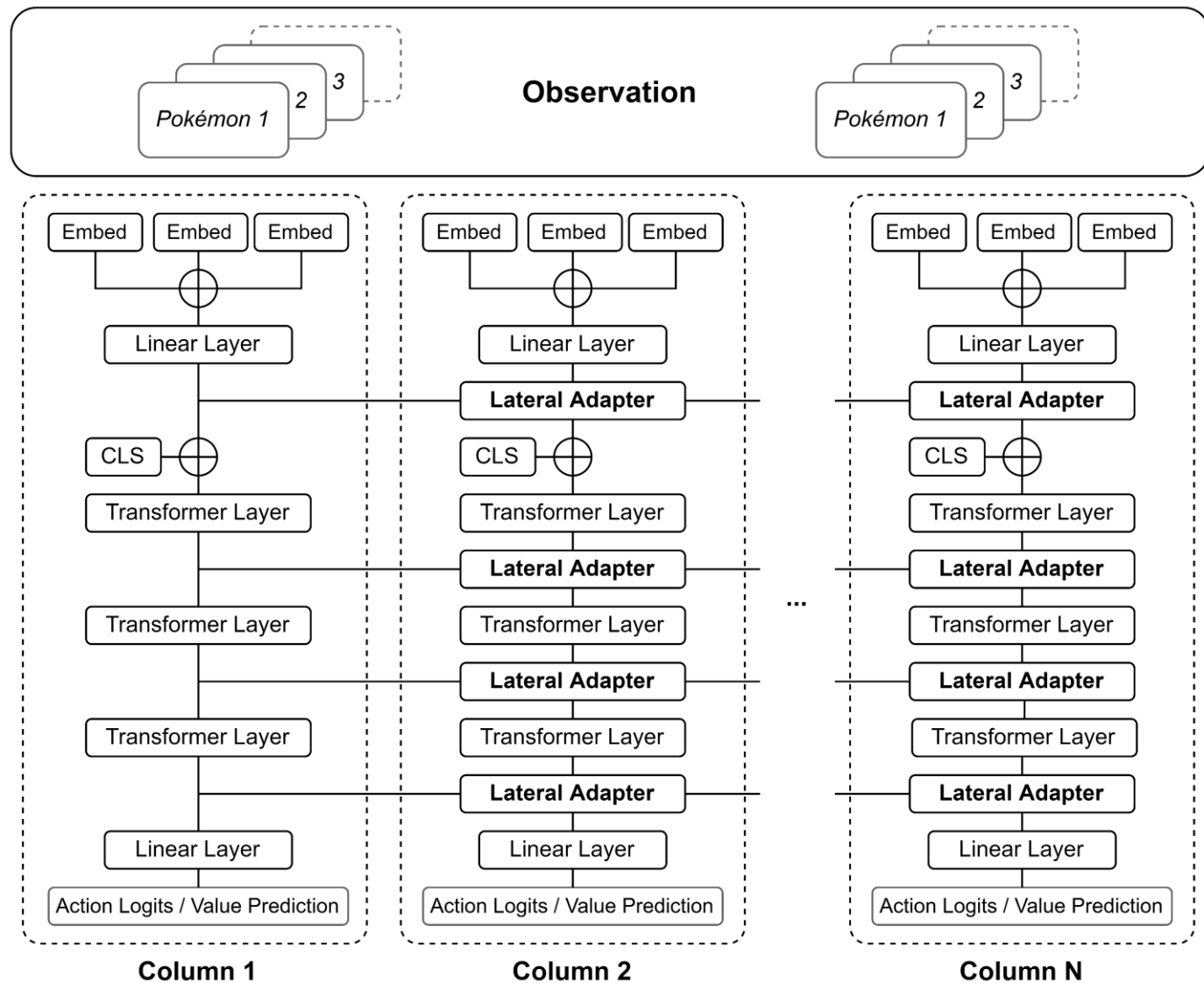
Methods - Continual Reinforcement Learning

- **Regularization methods:** penalize changes to important parameters.
 - *Elastic Weight Consolidation, EVCL, Synaptic Intelligence*
 - Policy Distillation: *Actor-Mimic, Distal*
- **Replay methods:** store and replay examples from previous tasks.
 - *CLEAR, EDER*
- **Parameter-isolation methods:** allocate separate parameters for tasks.
 - Progressive Architectures: *Progressive Neural Networks¹, PathNet, LoRA*
 - Specialized Optimization Methods: *ReDo, Continual Backprop, Tuned PPO²*

[1] Rusu et al. "Progressive neural networks." (2016)

[2] Dohare et al. "Loss of plasticity in deep continual learning." (2024)

Methods - Progressive Architecture



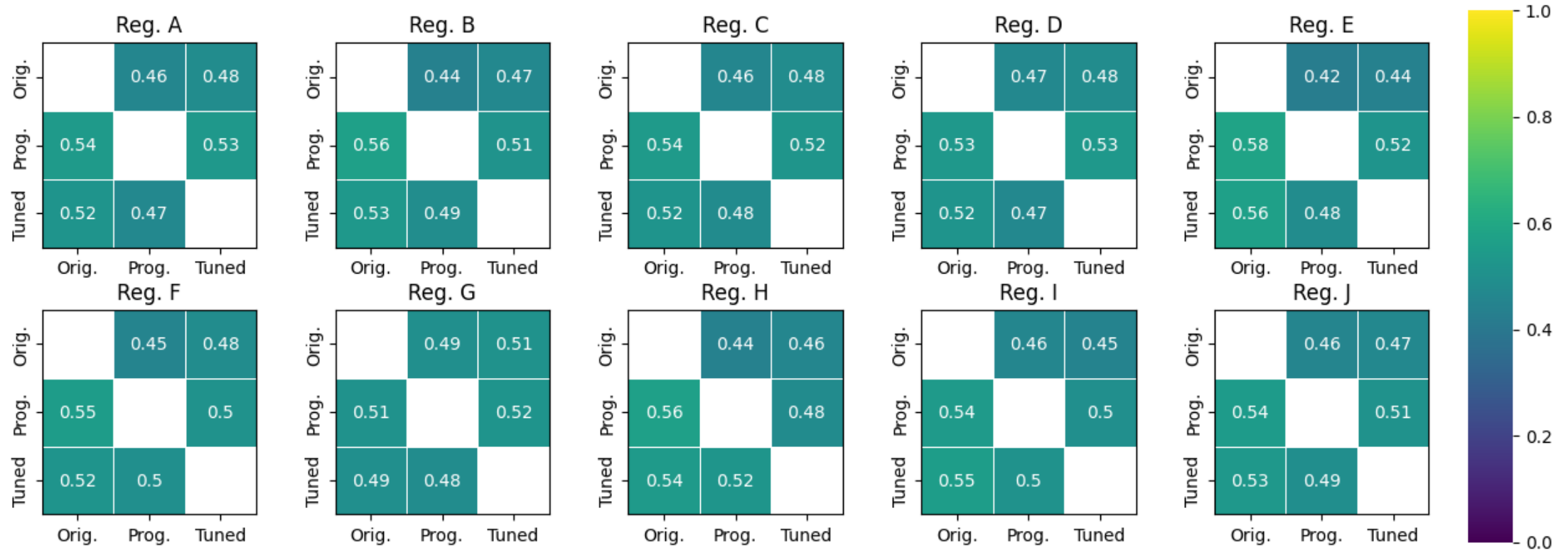
Methods - Tuned PPO

- Default β s of the Adam optimizer:
 - $\beta_1 = 0.9$
 - $\beta_2 = 0.999$
- Lyle et al.¹ showed that $\beta_1 \neq \beta_2$ leads to high loss of plasticity, instead suggesting $\beta_1 = \beta_2 = 0.99$, resulting in smaller, stable updates.
- Dohare et al.² used this tuned Adam optimizer + L2 regularization in PPO (**Tuned PPO**), and showed it behaved similarly to Continual Backprop.

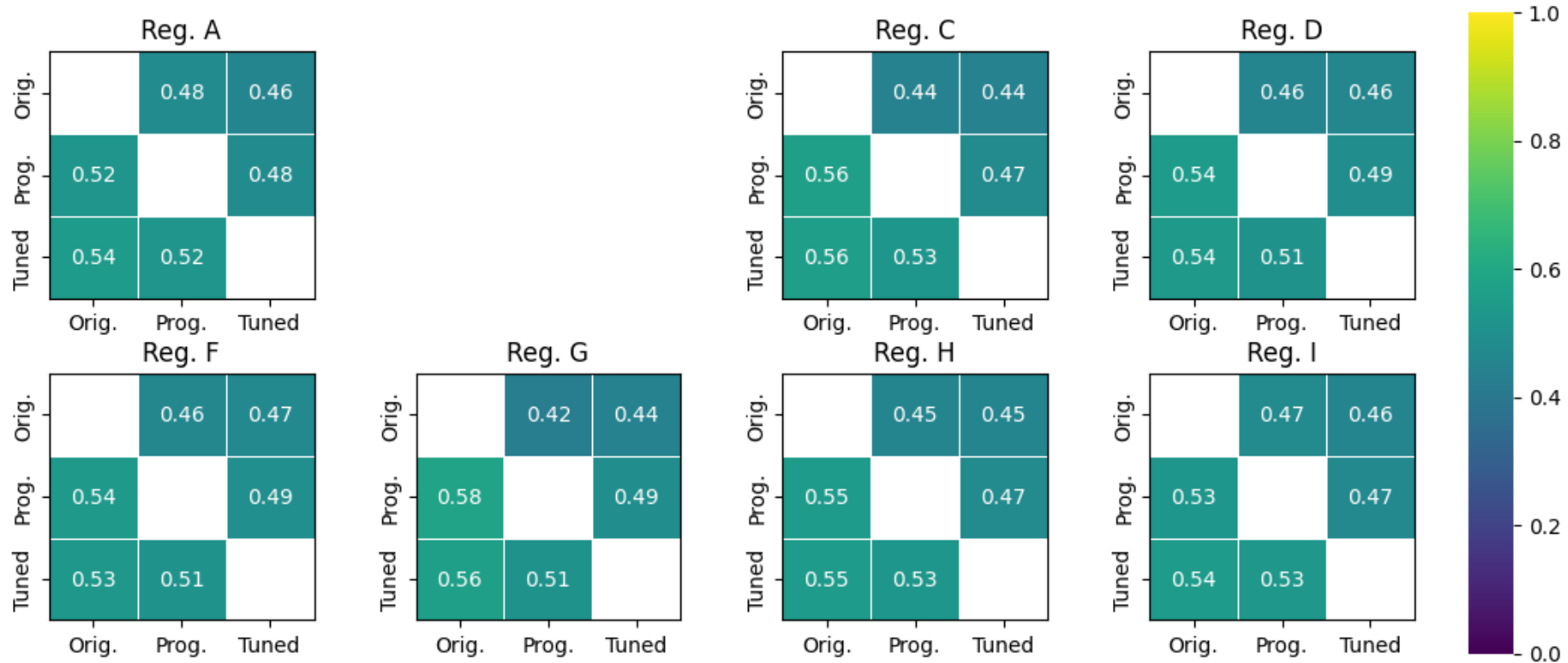
$$\lambda = 1\text{e-}5 \rightarrow \lambda = 1\text{e-}6$$

Stationary Experiments

Stationary Experiments

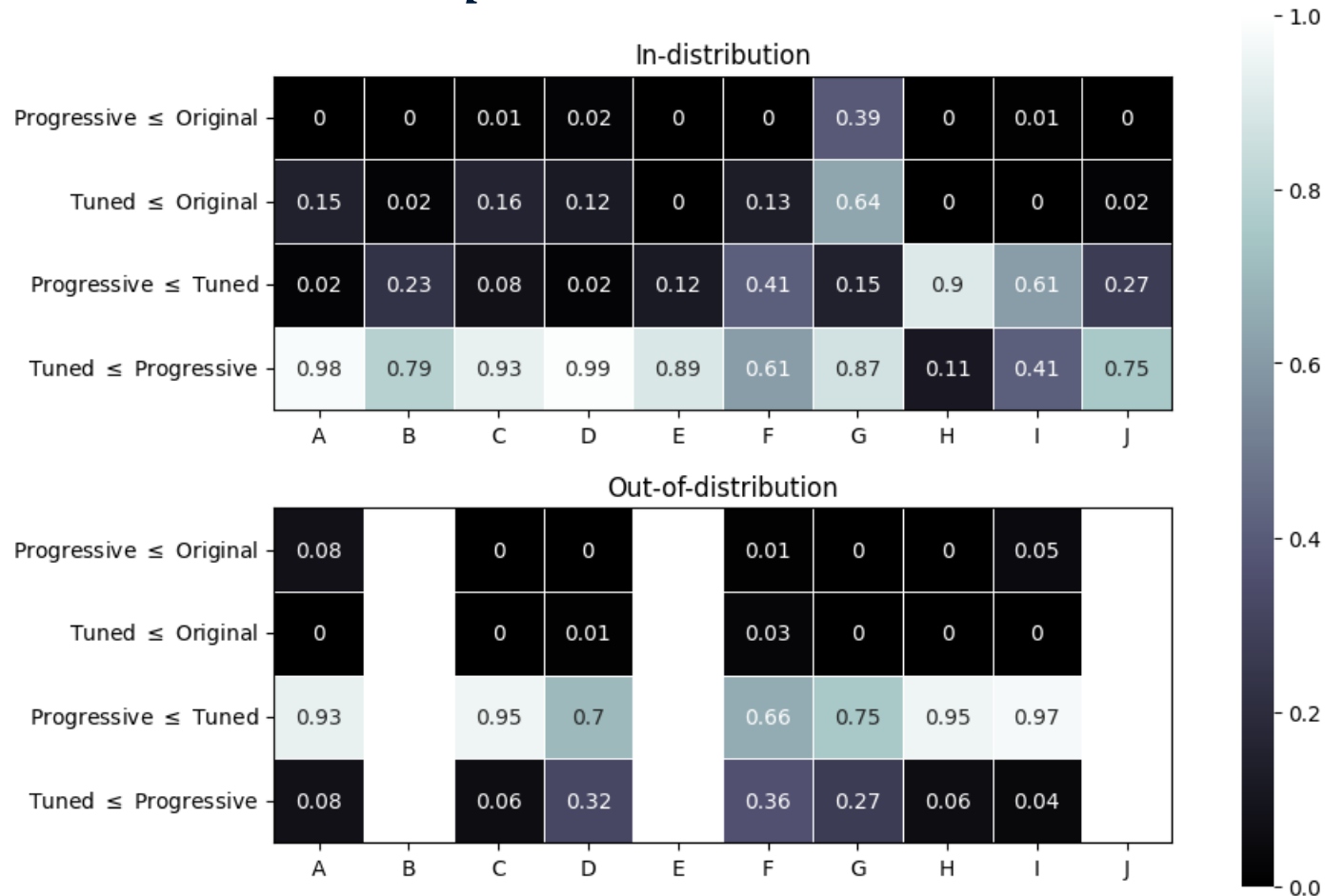


Stationary Experiments



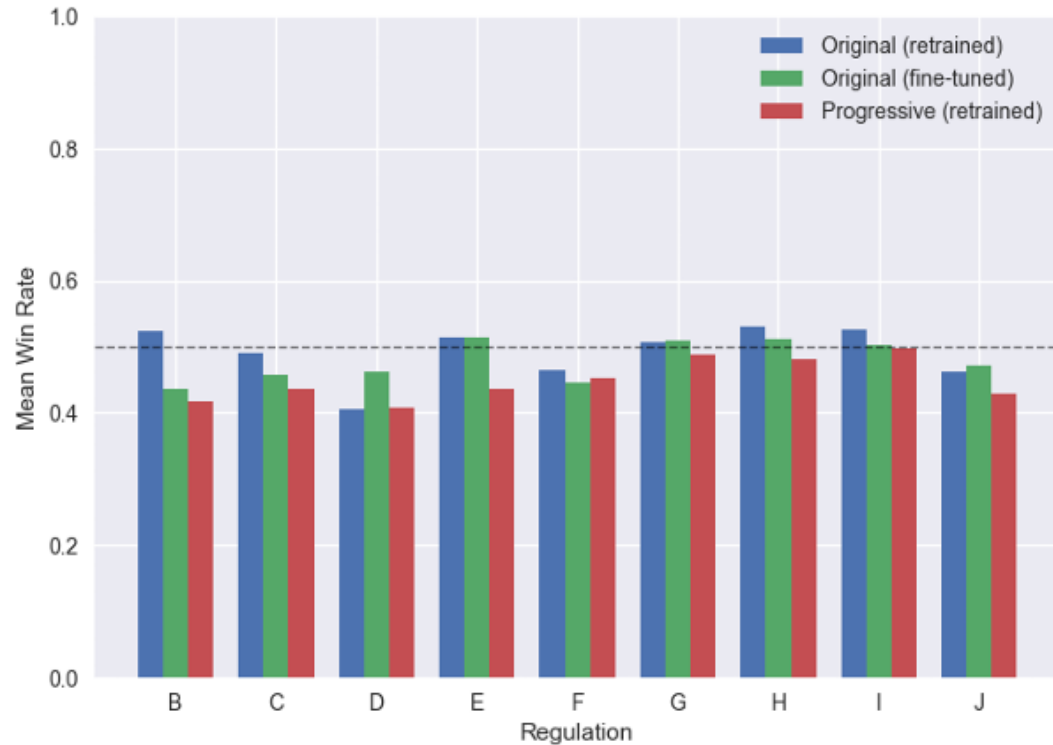
Stationary Experiments

- H_0 = An agent ϕ performs *equal or worse* than agent ψ
- → **Binomial Test**: $n = 1000, p \leq 0.5, \alpha = 0.05$

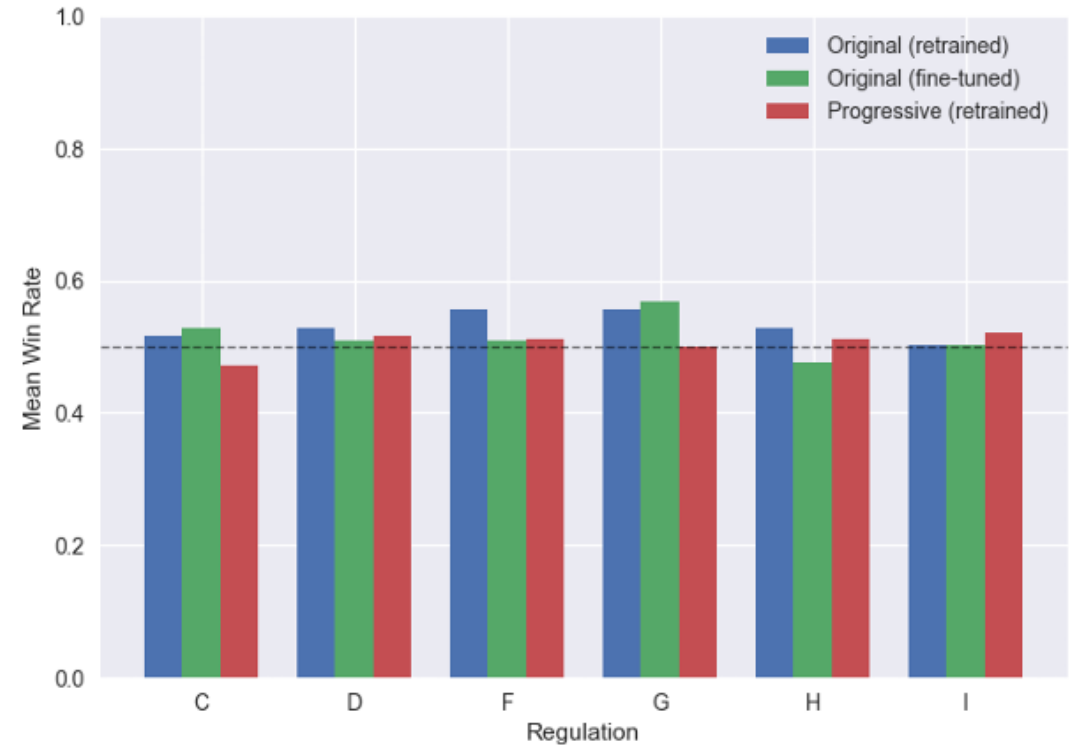


Non-Stationary Experiments

Non-Stationary Experiments

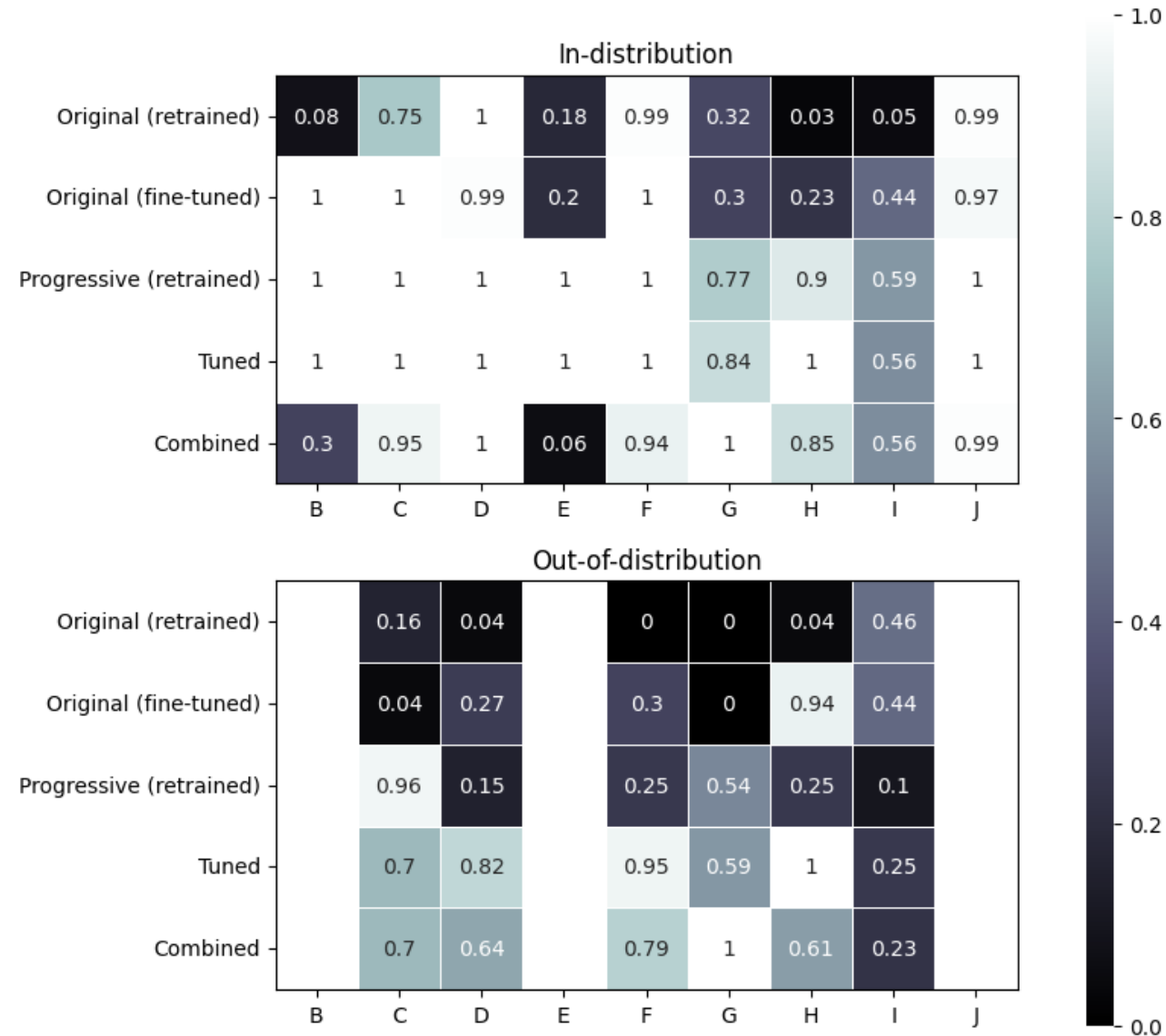


(a) In-distribution.

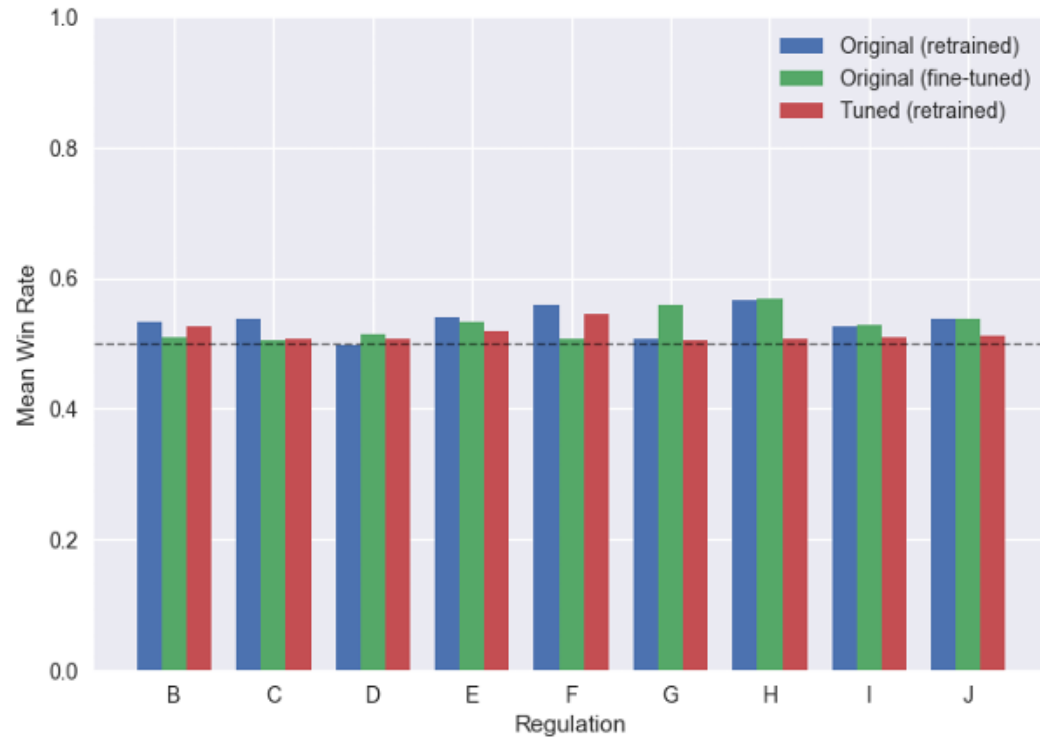


(b) Out-of-distribution.

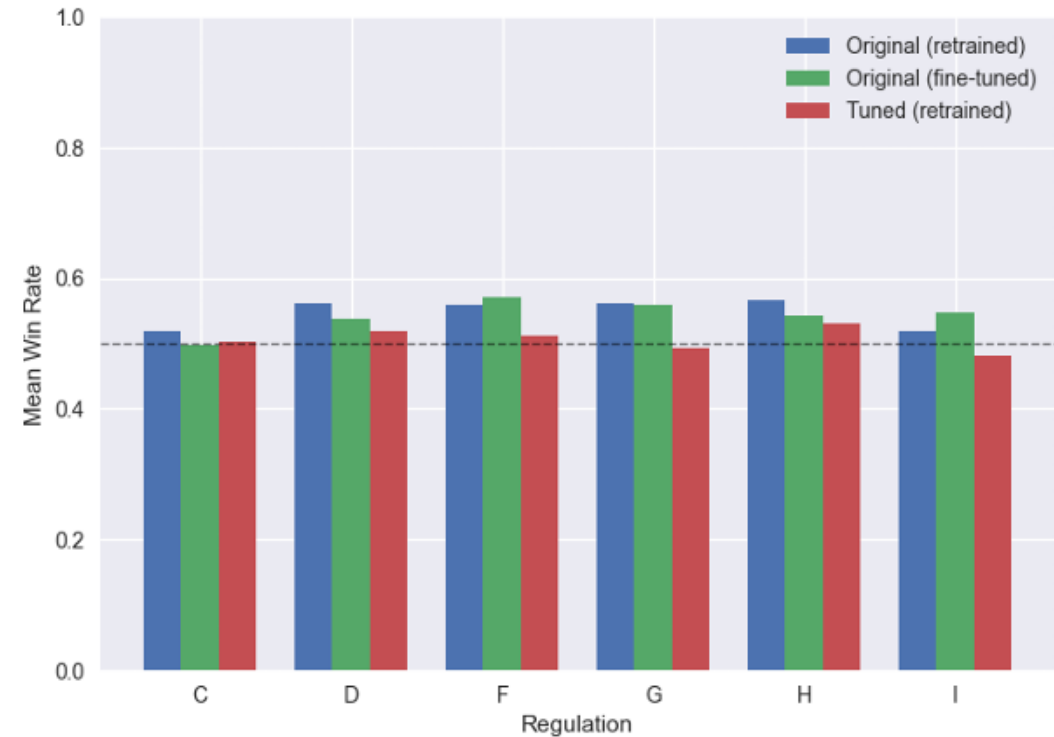
Non-Stationary Experiments



Non-Stationary Experiments

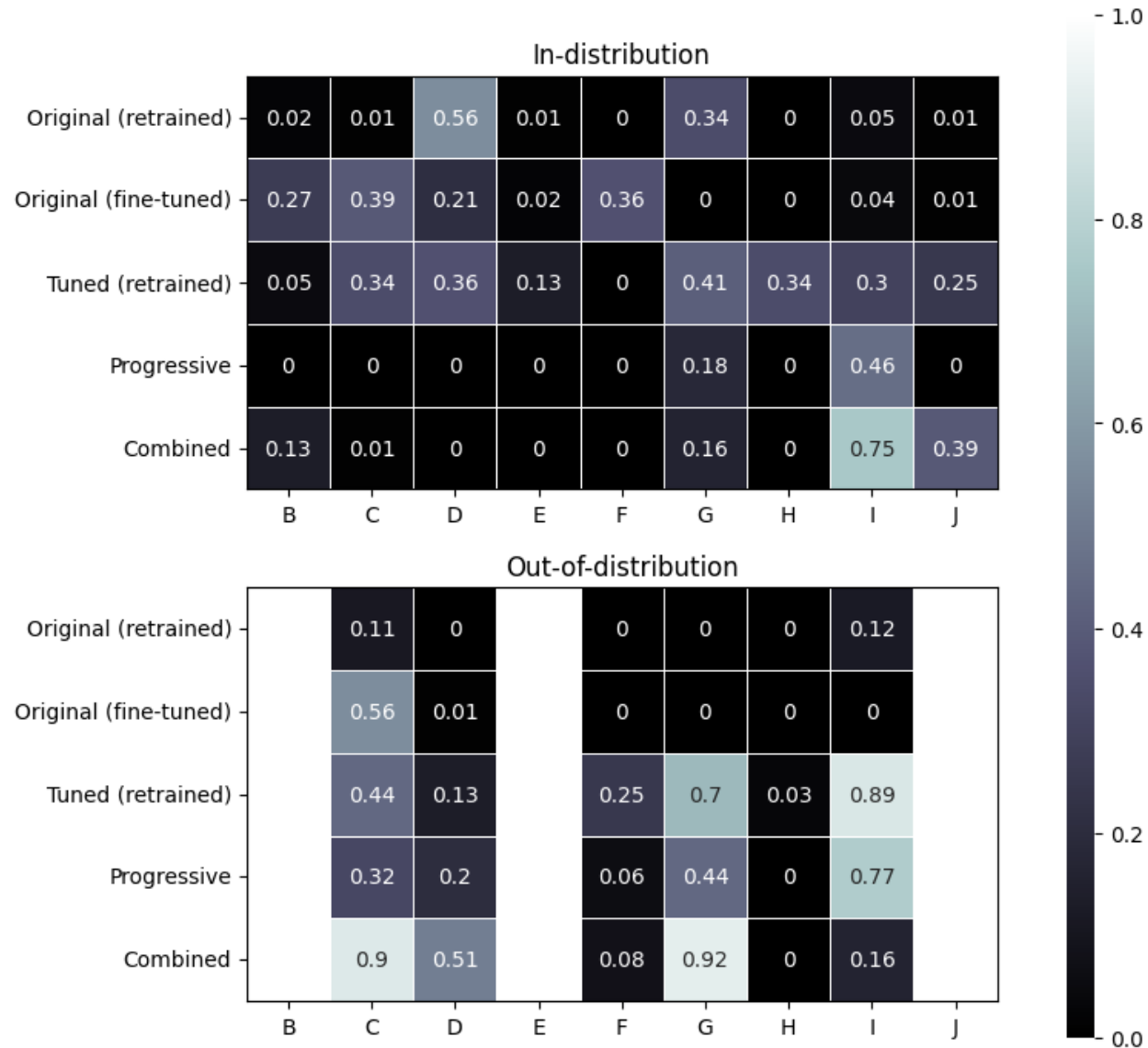


(a) In-distribution.

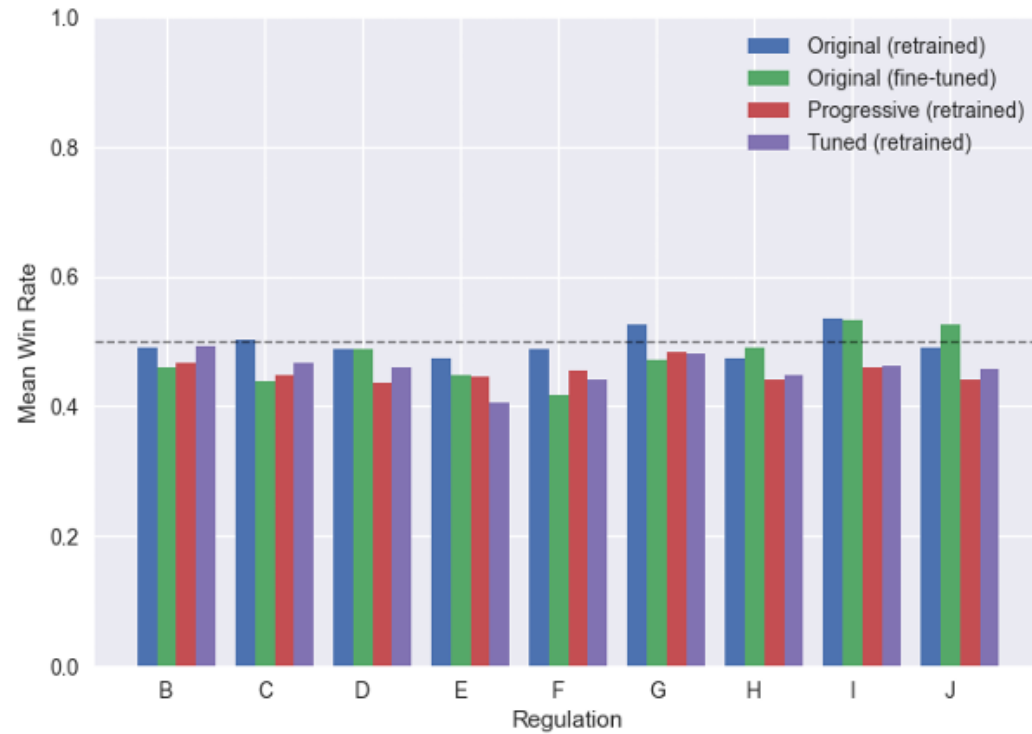


(b) Out-of-distribution.

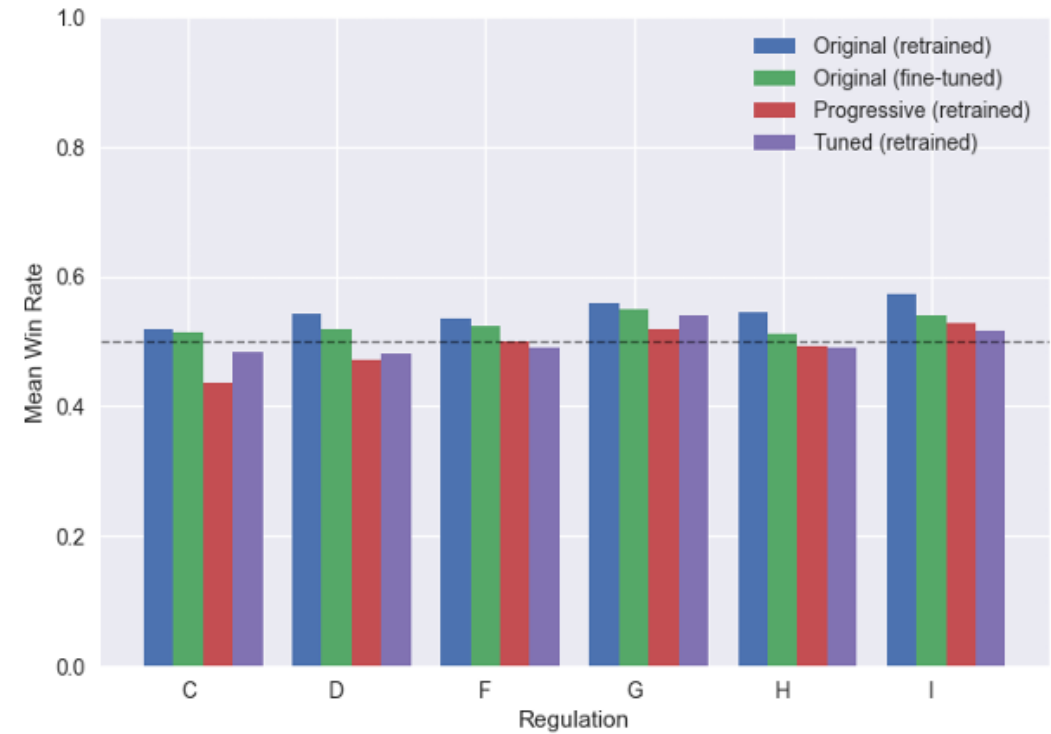
Non-Stationary Experiments



Non-Stationary Experiments

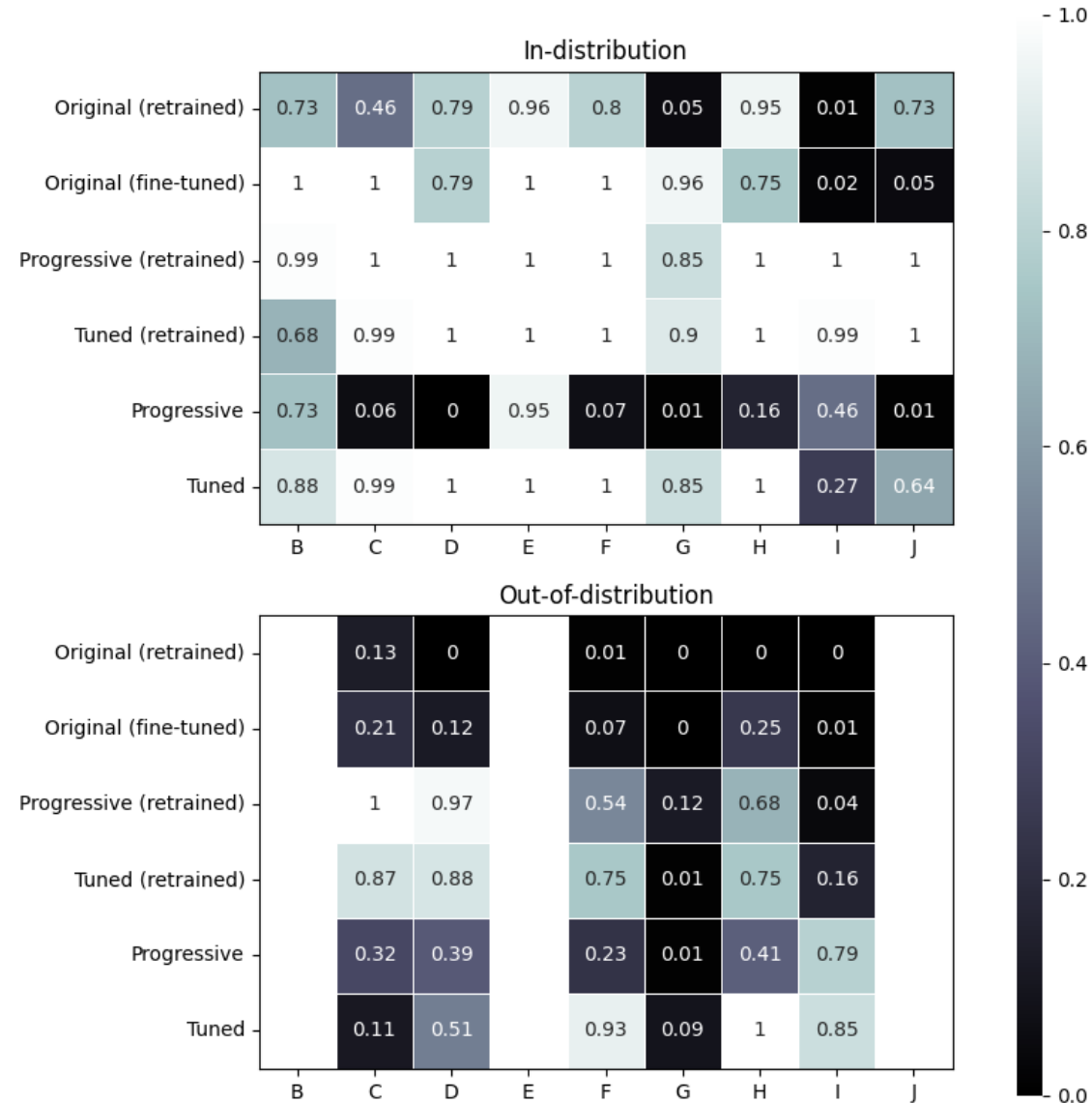


(a) In-distribution.



(b) Out-of-distribution.

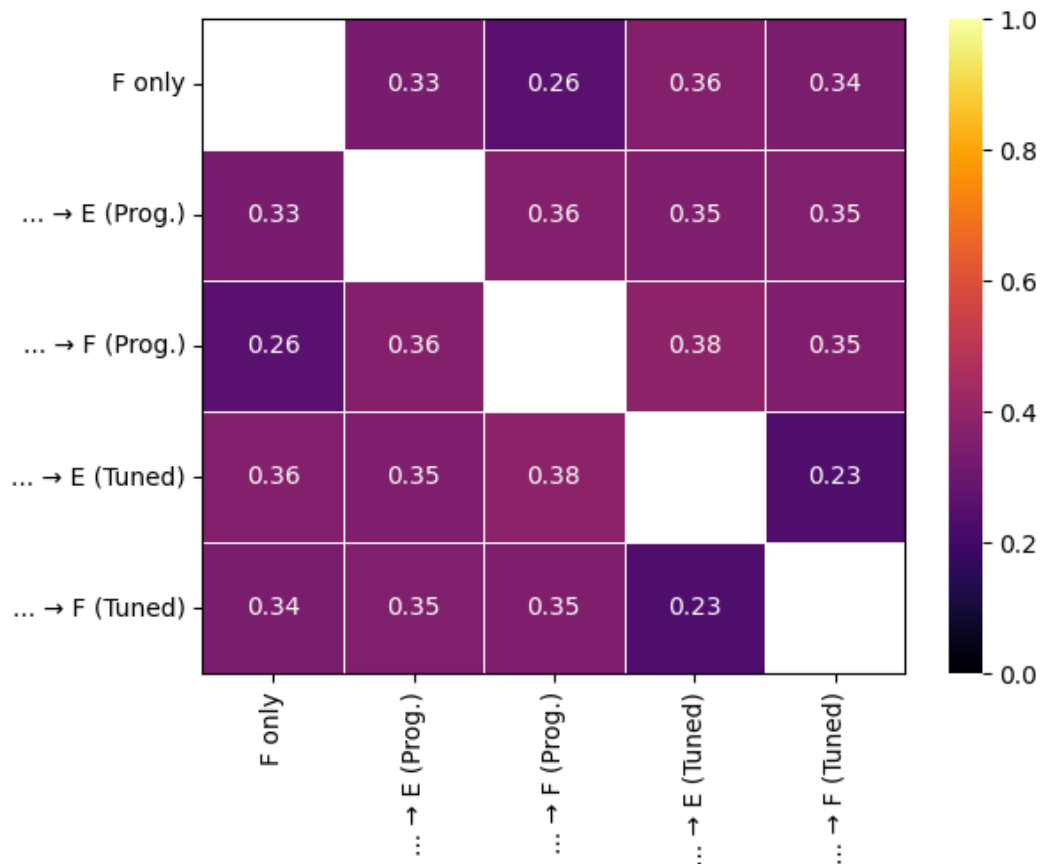
Non-Stationary Experiments



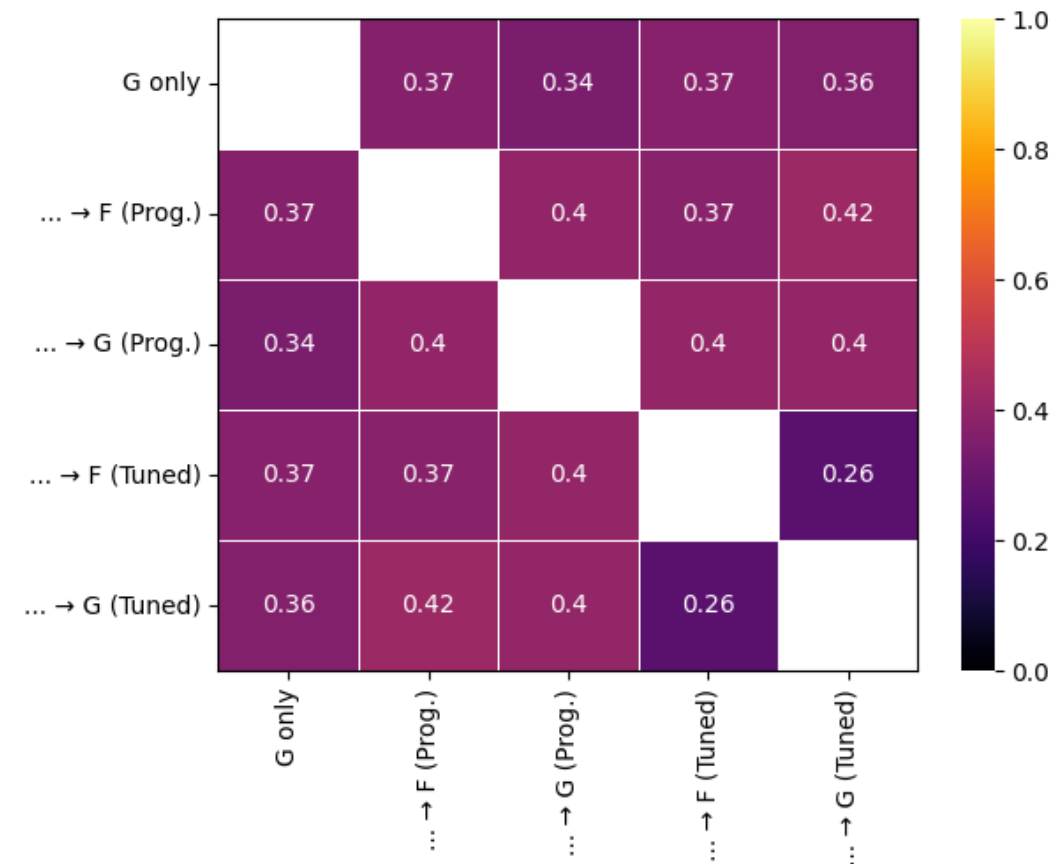
Analysis

Analysis

- Mean pairwise Jensen-Shannon distance between action probabilities:



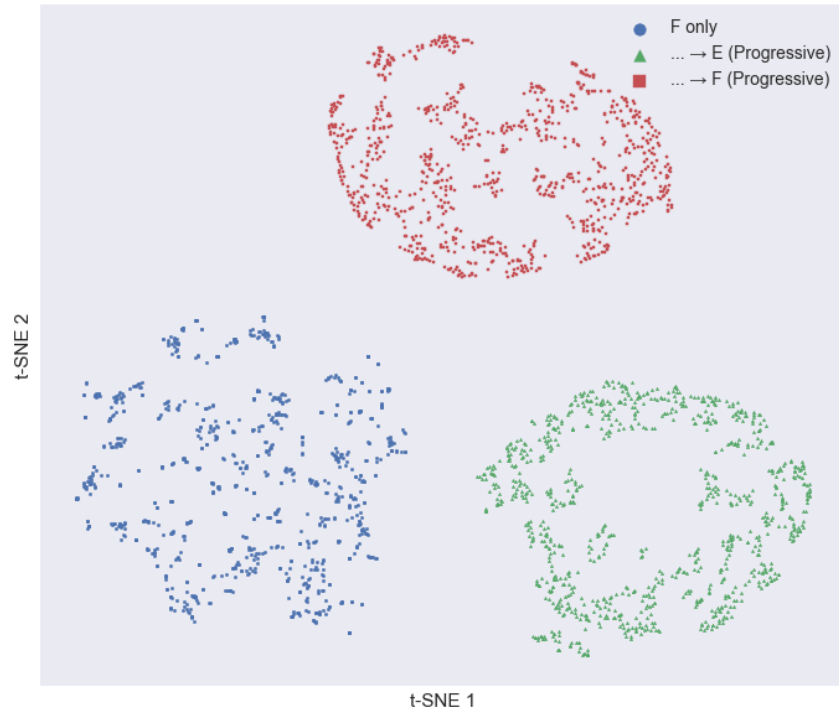
(Regulation **F**, in-distribution)



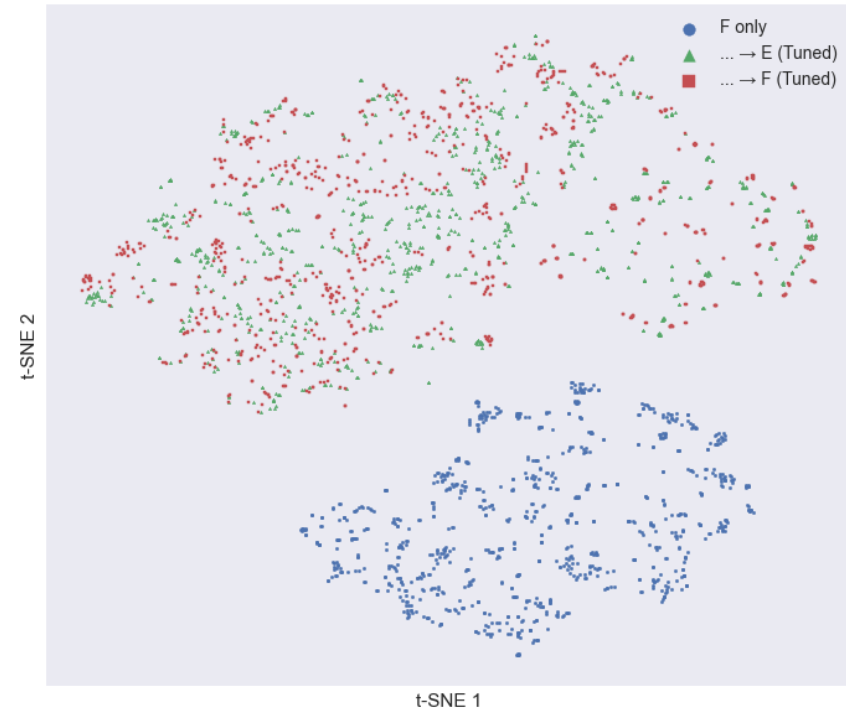
(Regulation **G**, in-distribution)

Analysis

- Distributions of action logits (t-SNE):



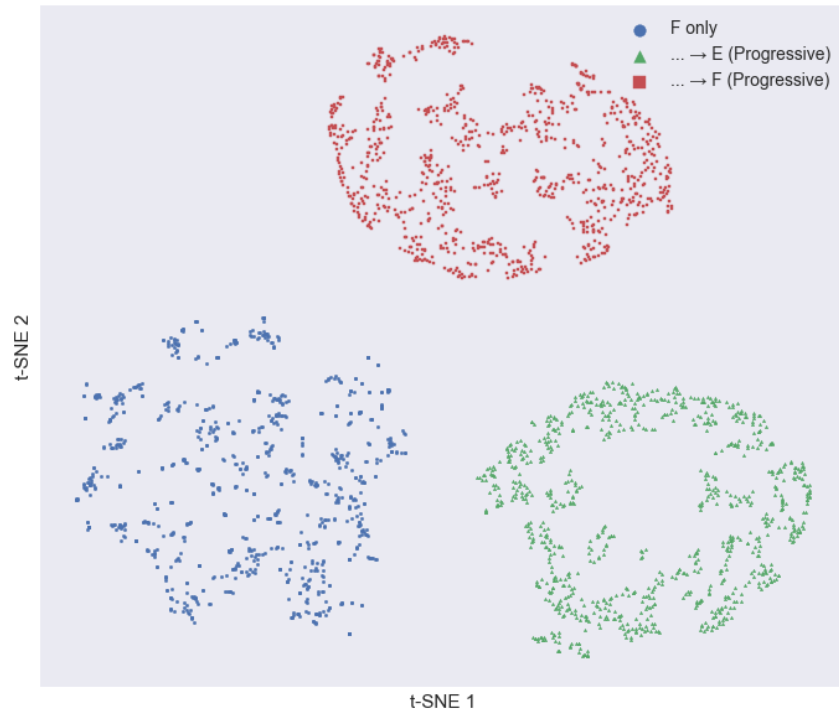
(Progressive, Regulation **F**, in-distribution)



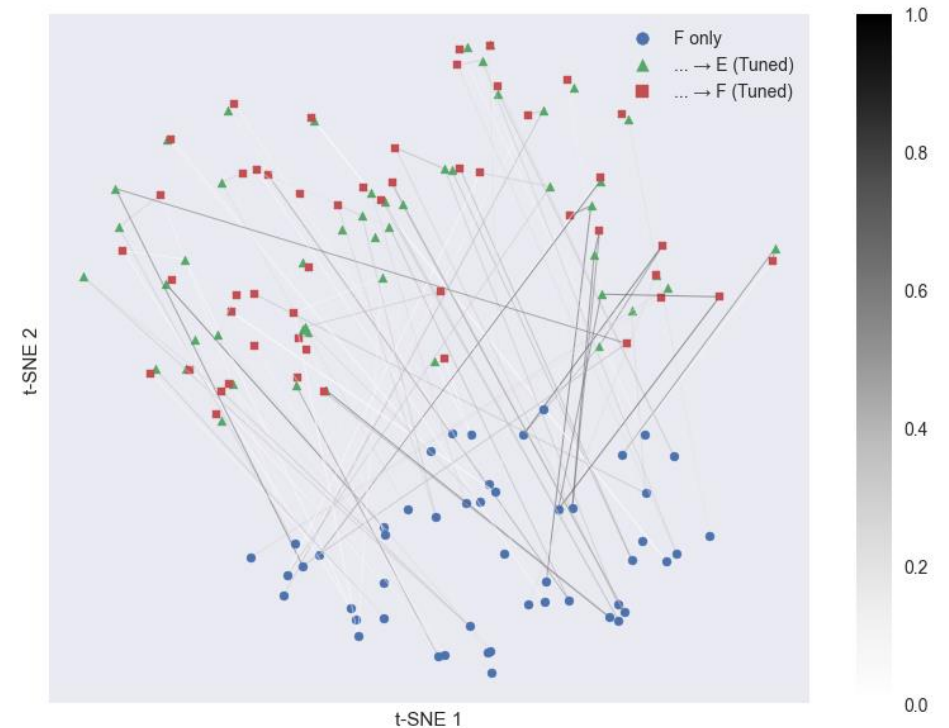
(Tuned, Regulation **F**, in-distribution)

Analysis

- Distributions of action logits (t-SNE):



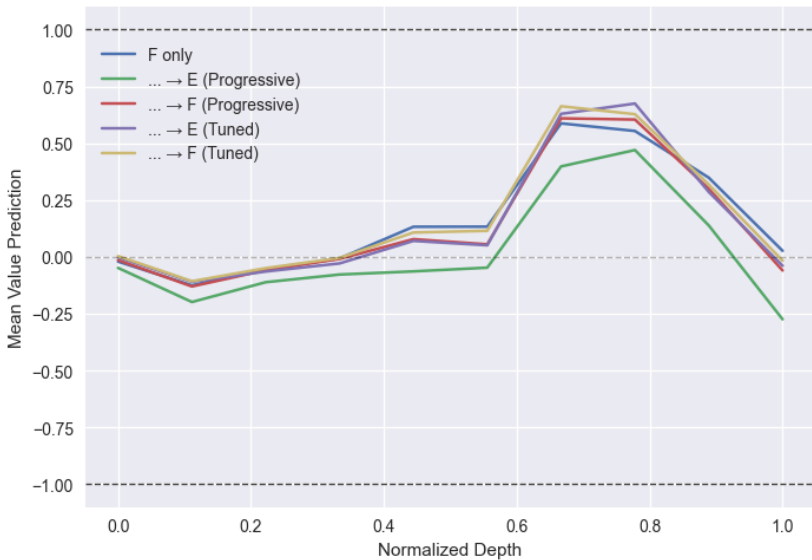
(Progressive, Regulation **F**, in-distribution)



(Tuned, Regulation **F**, in-distribution)

Preliminary Experiments - Analysis

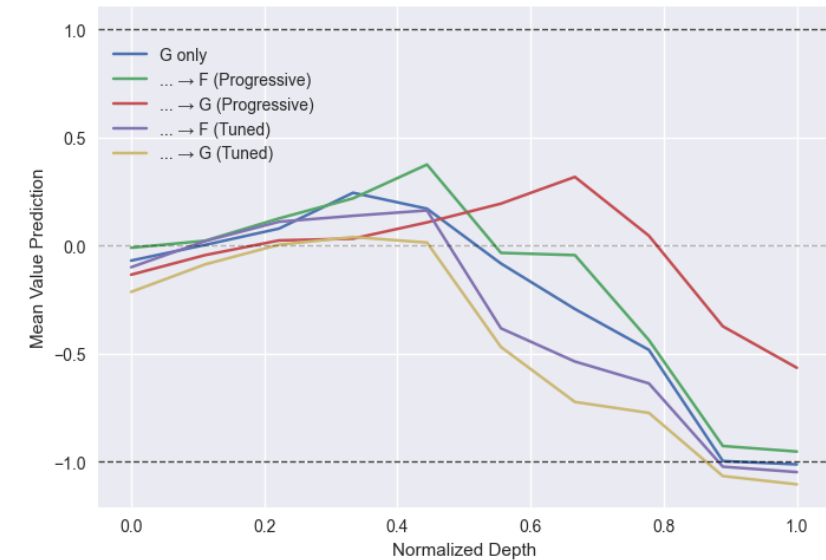
- Mean value prediction:



(Regulation **F**, in-distribution)



(Regulation **F**, out-of-distribution)



(Regulation **G**, in-distribution)

Conclusion

Conclusion

1. **What effect does non-stationarity have on the performance and generalization of reinforcement learning agents in VGC?**
 - Agents trained in one regulation are consistently worse than retrained agents in other regulations.
 - When evaluating for generalization, agents can match other agents trained on future regulations.

Conclusion

2. Can continual reinforcement learning methods facilitate effective knowledge transfer across game versions?

- Agents can transfer knowledge short-term using full fine-tuning.
- Our proposed network adjustments + tuned PPO outperform full fine-tuning, successfully prolonging the effective transfer.
- Our progressive architecture is unable to facilitate better knowledge transfer, but does exhibit better generalization.

Conclusion

3. **How do continual reinforcement learning-based agents compare to fully retrained agents in terms of performance, generalization, and training cost across game updates?**
 - All our methods are unable to consistently outperform retrained agents.
 - Retrained agents are superior in terms of performance, generalization, and training cost, though continual learning appears to still be beneficial.

Future Work

- Our experiments should be repeated on multiple seeds.
- Possible directions:
 - The value function
 - L2 regularization rate
 - Number of teams used during training
 - Hyperparameters
 - ReDo & Continual Backprop
 - Regularization- & Replay methods
 - Progressive architecture
 - Custom regulations
 - Other games
 - ...

Future Work

- We believe continual learning in non-stationary games is a severely understudied domain that shows great potential.
- Modern games, like VGC, have clearly defined rules, goals, and task boundaries.
- Non-stationary games could serve as natural benchmark for continual learning, facilitating a greater understanding of how machine learning agents can remain effective after deployment.

Questions?

Appendix

VGC-Bench Hyperparameters

Parameter	Value
Learning Rate	$1\text{e-}5 \times 0.1^{(1-p)}$
Discount Factor (γ)	1.0
GAE Lambda (λ)	0.95
Clip Range	0.2
Entropy Coefficient	0.001
Value Function Coefficient	0.5
Max Gradient Norm	0.5
Number of Steps per Update	24×128
Batch Size	512
Number of Epochs	10
Total Timesteps	5 013 504

Table 2.2: Hyperparameters used for all reinforcement learning experiments in VGC-Bench [10].